

DOCUMENTO TÉCNICO

Guía de Despliegue

WodenTrack

Backend NestJS · Frontend Vue (Vite) · Variables de entorno por ambiente

VERSIÓN	FECHA	AUTOR	STACK
2.0.1	Mayo 2026	E. Agudelo	NestJS + Vue + Vite + PM2 + IIS

1 Ambientes disponibles

El proyecto maneja tres ambientes, cada uno con su propio archivo `.env` y sus propias URLs de API.

AMBIENTE	ARCHIVO BACKEND	ARCHIVO FRONTEND	PUERTO BACKEND	URL API
DEVELOPMENT	<code>.env.development</code>	<code>.env.development</code>	8082	localhost:8082
QA	<code>.env.qa</code>	<code>.env.qa</code>	8085	18.217.246.39:8085
PRODUCTION	<code>.env.production</code>	<code>.env.production</code>	8082	18.217.246.39:8082



Los archivos `.env.*` están en `.gitignore` y **nunca se suben al repositorio**. Deben crearse manualmente en cada servidor.

2 Cómo funciona la selección de ambiente

BACKEND — NESTJS

En `src/app.module.ts`, el `ConfigModule` carga el archivo `.env` según la variable `NODE_ENV`:

```
ConfigModule.forRoot({
  isGlobal: true,
  envFilePath: [
    `.env.${process.env.NODE_ENV || 'development'}`,
    '.env', // fallback si no existe el específico
  ],
})
```

```
  ],
  }),
```

La variable `NODE_ENV` se inyecta mediante `cross-env` en los scripts de `package.json`.

FRONTEND — VUE + VITE

Vite carga automáticamente el archivo `.env.[mode]` según el flag `--mode` que se pase al compilar. Las variables deben tener el prefijo `VITE_` para estar disponibles en el código.

```
# En el código Vue se accede así:
import.meta.env.VITE_API_URL
```

3 Contenido de los archivos .env

Estos archivos deben crearse manualmente en el servidor. **No están en el repositorio git.**

BACKEND — .ENV.DEVELOPMENT

```
PORT=8082
APP_VERSION=2.0.1

# ODOO
ODOO_URL=https://woden.odoo.com
ODOO_DB=wdsit-wodoo-main-9547586
ODOO_USER=woden.botit@woden.com.co
ODOO_PASS=*****

WEBCONFIG_PATH=C:\ruta\local\web.config

# BASE DE DATOS SQL SERVER
DB_TYPE=mssql
DB_HOST=3.133.217.145
DB_PORT=1433
DB_USER=userwebapp
DB_PASS=*****
DB_NAME=WodenTrackTest

# MAIL
MAIL_HOST=smtp.office365.com
MAIL_PORT=587
MAIL_USER=eagudelo@woden.com.co
MAIL_PASS=*****
MAIL_ALERT_TO=eagudelo@woden.com.co
MAIL_ALERTS_ENABLED=false
```

```
# AWS S3
AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=*****
AWS_SECRET_ACCESS_KEY=*****
AWS_S3_BUCKET=novedades-woden-track
```

BACKEND — .ENV.QA

```
PORT=8085
APP_VERSION=2.0.1

# ODOO — QA
ODOO_URL=https://woden-qa011-26713137.dev.odoo.com
ODOO_DB=woden-qa011-26713137
ODOO_USER=woden.botit@woden.com.co
ODOO_PASS=*****

WEBCONFIG_PATH=C:\inetpub\wwwroot\WodenTrackTest\dist\web.config

# (resto igual que development)
DB_TYPE=mssql · DB_HOST=3.133.217.145 · DB_PORT=1433 · DB_NAME=WodenTrackTest
MAIL_*=igual que development
AWS_*=igual que development
```

BACKEND — .ENV.PRODUCTION

```
PORT=8082
APP_VERSION=2.0.1

# ODOO — PRODUCCIÓN
ODOO_URL=https://woden.odoo.com
ODOO_DB=wdsit-wodoo-main-9547586
ODOO_USER=woden.botit@woden.com.co
ODOO_PASS=*****

WEBCONFIG_PATH=C:\inetpub\wwwroot\WodenTrackTest\dist\web.config

# (resto igual que development)
```

FRONTEND — .ENV.DEVELOPMENT / .ENV.QA / .ENV.PRODUCTION

```
# .env.development
VITE_API_URL=http://localhost:8082/usuarios
```

```
# .env.qa
VITE_API_URL=http://18.217.246.39:8085/usuarios

# .env.production
VITE_API_URL=http://18.217.246.39:8082/usuarios
```

4 Scripts de compilación y arranque

BACKEND — PACKAGE.JSON

```
"scripts": {
  "start": "cross-env NODE_ENV=development nest start",
  "start:dev": "cross-env NODE_ENV=development nest start --watch",
  "start:prod": "cross-env NODE_ENV=production node dist/main",
  "start:qa": "cross-env NODE_ENV=qa node dist/main",
  "build": "nest build",
  "build:prod": "cross-env NODE_ENV=production nest build",
  "build:qa": "cross-env NODE_ENV=qa nest build"
}
```

FRONTEND — PACKAGE.JSON

```
"scripts": {
  "dev": "vite --mode development", // localhost
  "build": "vite build --mode production", // producción
  "build:qa": "vite build --mode qa", // QA
  "build:dev": "vite build --mode development", // development build
  "preview": "vite preview"
}
```

5 Archivo ecosystem.config.js (PM2)

Este archivo le dice a PM2 con qué `NODE_ENV` arrancar el backend. Se crea en la carpeta raíz del backend en el servidor.

```
// ecosystem.config.js — servidor QA
module.exports = {
  apps: [
    {
      name: 'WodenTrackTest',
      script: 'dist/main.js',
```

```
    env: {
      NODE_ENV: 'qa',          // carga .env.qa
    },
  },
],
};

// ecosystem.config.js – servidor PRODUCCIÓN
module.exports = {
  apps: [
    {
      name: 'WodenTrack',
      script: 'dist/main.js',
      env: {
        NODE_ENV: 'production', // carga .env.production
      },
    },
  ],
};
```

6 Despliegue del Backend (paso a paso)

1 Compilar en tu máquina local

```
# Para QA:
npm run build:qa

# Para producción:
npm run build:prod
```

Esto genera la carpeta **dist/** con el código compilado.

2 Subir el código al servidor

Copia la carpeta **dist/** y el **package.json** al servidor. También el **ecosystem.config.js** si no existe.

3 Instalar dependencias en el servidor (solo si package.json cambió)

```
cd C:\ruta\del\backend
npm install --omit=dev
```

4 Verificar que el .env del ambiente exista en el servidor

```
# Verificar que existe:
ls .env.qa          # o .env.production

# Si no existe, crearlo (ver sección 3 de esta guía)
```

5 Reiniciar con PM2

```
# Si ya existe el proceso en PM2:
pm2 restart WodenTrackTest

# Si es la primera vez (o se borró el proceso):
pm2 stop WodenTrackTest
pm2 delete WodenTrackTest
pm2 start ecosystem.config.js
pm2 save
```

6 Verificar que arrancó correctamente

```
pm2 list                # debe mostrar "online"
pm2 logs WodenTrackTest --lines 20 # buscar "🚀 Servidor NestJS corriendo"
netstat -ano | findstr :8085      # debe aparecer LISTENING
```

7 Despliegue del Frontend — IIS (paso a paso)

i El frontend **no necesita archivos .env en el servidor**. La URL del API queda embebida en el **dist/** al momento de compilar. IIS solo sirve archivos estáticos.

1 Compilar en tu máquina local con el ambiente correcto

```
# Para QA:
cd frontend
npm run build:qa

# Para producción:
npm run build
```

Genera la carpeta **frontend/dist/** con HTML, JS y CSS listos.

2 Verificar que la URL quedó correcta en el build

En PowerShell, buscar la URL dentro del JS generado:

```
Select-String -Path "dist\assets\*.js" -Pattern "8082|8085" | Select-Object -First 3
```

Debe mostrar la URL del ambiente que compilaste.

3 Subir el dist/ al servidor y reemplazar en IIS

Copia el contenido de **dist/** a la carpeta que IIS está sirviendo (por ejemplo **C:\inetpub\wwwroot\WodenTrackTest\dist**). Reemplaza todos los archivos.

4 El web.config de IIS ya debe existir con el modo history de Vue Router

Si no existe, créalo en la carpeta raíz del sitio IIS:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="Handle History Mode" stopProcessing="true">
          <match url="(.*)" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

8 Comandos PM2 de referencia rápida

COMANDO	DESCRIPCIÓN
<code>pm2 list</code>	Ver todos los procesos y su estado
<code>pm2 show WodenTrackTest</code>	Ver detalle del proceso (puerto, NODE_ENV, etc.)
<code>pm2 env <id></code>	Ver variables de entorno del proceso (usar el id numérico de pm2 list)
<code>pm2 restart WodenTrackTest</code>	Reiniciar el proceso (después de actualizar código)
<code>pm2 logs WodenTrackTest --lines 50</code>	Ver últimas 50 líneas del log
<code>pm2 start ecosystem.config.js</code>	Arrancar desde el archivo de configuración

COMANDO	DESCRIPCIÓN
<code>pm2 save</code>	Guardar la lista de procesos (persiste tras reinicio del servidor)
<code>pm2 startup</code>	Hacer que PM2 arranque automáticamente con Windows
<code>pm2 stop WodenTrackTest</code>	Detener el proceso
<code>pm2 delete WodenTrackTest</code>	Eliminar el proceso de PM2

9 Diagnóstico de problemas comunes

EADDRINUSE — PUERTO YA EN USO

Hay una instancia anterior del backend corriendo en el mismo puerto.

```
# Ver qué proceso ocupa el puerto:
netstat -ano | findstr :8085
# Reiniciar limpiamente con PM2:
pm2 restart WodenTrackTest
```

EACCES — PERMISO DENEGADO EN EL PUERTO

Windows bloquea puertos por debajo de 1024 o algunos reservados por IIS/Hyper-V. Solución: cambiar el puerto en el `.env` a uno libre (ej. 8085, 8087, 8088) y reiniciar.

NODE_ENV NO SE APLICA (CARGA .ENV INCORRECTO)

```
# Verificar qué NODE_ENV tiene el proceso:
pm2 env <id> # buscar NODE_ENV en la lista

# Si no tiene, recrear con ecosystem.config.js:
pm2 delete WodenTrackTest
pm2 start ecosystem.config.js
pm2 save
```

405 METHOD NOT ALLOWED EN EL LOGIN

El frontend apunta a un puerto diferente al que corre el backend. Verificar que la URL en el `dist/` compilado coincida con el puerto real del backend.



Para confirmar que el backend responde: abrir en el navegador `http://IP:PUERTO/usuarios/version`. Si retorna JSON con la versión, el backend está vivo.

WodenTrack Pro · Guía de Despliegue · Mayo 2026
Documento de uso interno — E. Agudelo